



Function Declaration

Function Expression

Arrow Function

Anonymous Function

Named Function Expression

Immediately Invoked Function Expression (IIFE)

Constructor Function

Generator Function

Async Function

Function as a Method

Getter & Setter Functions

Higher-Order Functions

Rest Parameter Functions

Default Parameter Functions

Here's a concise overview of the key types of JavaScript functions you listed, with brief explanations and syntax examples for each:

1. Function Declaration

- **Definition:** Named, hoisted function.

```
function add(a, b) {  
  return a + b;  
}
```

2. Function Expression

- **Definition:** Function assigned to a variable, not hoisted.

```
const subtract = function(a, b) {  
  return a - b;  
};
```

3. Arrow Function

- **Definition:** ES6 syntax; lexical `this`; concise form.

```
const multiply = (a, b) => a * b;
```

4. Anonymous Function

- **Definition:** Function without a name, often used for callbacks.

```
setTimeout(function() {  
  console.log('Hello!');  
}, 1000);
```

5. Named Function Expression

- **Definition:** Function expression with a name, useful for recursion.

```
const factorial = function fact(n) {  
  return n <= 1 ? 1 : n * fact(n - 1);  
};
```

6. Immediately Invoked Function Expression (IIFE)

- **Definition:** Function executed immediately after its definition.

```
(function() {  
  console.log('IIFE ran!');  
})();
```

7. Constructor Function

- **Definition:** Function designed to create objects (used with `new`).

```
function Person(name) {  
  this.name = name;  
}  
const user = new Person('Raj');
```

8. Generator Function

- **Definition:** Function that can pause and resume execution, returns an iterator.

```
function* genNumbers() {  
  yield 1;  
  yield 2;  
}
```

9. Async Function

- **Definition:** Declared with `async`, uses `await` for asynchronous code.

```
async function fetchData() {  
  const response = await fetch('url');  
  return response.json();  
}
```

10. Function as a Method

- **Definition:** Function defined inside an object, accessed via property.

```
const calculator = {  
  add(a, b) { return a + b; }  
};
```

11. Getter & Setter Functions

- **Definition:** Special methods for property access.

```
const obj = {  
  _value: 10,  
  get value() { return this._value; },  
  set value(val) { this._value = val; }  
};
```

12. Higher-Order Functions

- **Definition:** Functions taking other functions as arguments or returning them.

```
function map(arr, fn) {  
  return arr.map(fn);  
}
```

13. Rest Parameter Functions

- **Definition:** Function using `...args` to accept varying number of arguments.

```
function sum(...nums) {  
  return nums.reduce((a, b) => a + b);  
}
```

14. Default Parameter Functions

- **Definition:** Function with default parameter values.

```
function greet(name = 'Guest') {  
  console.log('Hello, ' + name);  
}
```

Use these examples for quick reference and implementation in JavaScript code. Let me know if you want detailed differences or more advanced usage!